Acuity Analytics — Cloud Engineer Interview Preparation

AWS • Azure • Terraform • CI/CD • ArgoCD • Helm • Kubernetes • Security • Observability • DR • Databricks • Kafka

2. AWS + Azure

Q2. What is your experience with both AWS and Azure?

> “I have worked across both platforms, although my recent project was more Azure-focused. On Azure, I've worked with services around Synapse, networking, private endpoints, monitoring, identity and Azure DevOps. On AWS, I've worked with EC2, S3, VPC, IAM, Route 53, Lambda and CloudWatch.
>
> From an engineering perspective, I try to keep the architecture cloud-agnostic where possible. Terraform helps standardize infrastructure, while CI/CD and Kubernetes provide consistency across environments.”

Q3. How would you design a multi-cloud environment?

> “I would first separate the common platform layer from cloud-specific services. Networking, IAM, monitoring, deployment standards and Terraform modules would have standardized patterns, while AWS- and Azure-specific implementations would remain modular.
>
> For example, AWS would use VPC, IAM and CloudWatch, while Azure would use VNet, Entra ID and Azure Monitor. Kubernetes and application deployment standards can remain consistent using Helm and ArgoCD.
>
> I would also define centralized logging, security controls, tagging, cost controls and DR requirements across both clouds.”

---

3. AWS Networking

Q4. Explain AWS VPC architecture.

> “A VPC provides the isolated network boundary. I normally design it across multiple Availability Zones for resilience. Public subnets host components that require internet exposure, while private subnets are used for application and database workloads.
>
> Internet-facing traffic goes through an Internet Gateway, while private workloads can use NAT Gateway for controlled outbound access. Security Groups provide stateful instance-level filtering, while Network ACLs provide subnet-level stateless filtering.
>
> Route tables determine traffic paths, and VPC endpoints can provide private access to AWS services such as S3 without traversing the public internet.”

Q5. Security Group vs NACL?

> “Security Groups are stateful and attached to resources such as EC2 instances or ENIs. If inbound traffic is allowed, the response traffic is automatically allowed.

>

NACLs are stateless and operate at subnet level, so inbound and outbound rules need to be explicitly configured.

>

I generally use Security Groups for workload-level security and NACLs as an additional subnet-level control."

Q6. How would you troubleshoot an EC2 instance that cannot access the internet?

"I would troubleshoot layer by layer:

>

1. Check the instance IP and subnet.
2. Verify the route table has the correct default route.
3. Confirm the subnet is public if direct internet access is expected.
4. Verify Internet Gateway attachment.
5. If it's private, verify NAT Gateway and its route.
6. Check Security Group outbound rules.
7. Check NACL inbound and outbound rules.
8. Validate DNS resolution.
9. Finally test connectivity using curl, telnet or appropriate network tools.

>

I would also check recent infrastructure changes because networking failures frequently come from route, security or Terraform changes."

---

4. Azure Networking

Q7. Explain Azure VNet architecture.

"An Azure VNet provides the private networking boundary. I typically divide it into subnets based on workload and security requirements. NSGs control traffic at subnet or NIC level.

>

For private connectivity, I would use Private Endpoints where supported, and connectivity between networks can be established using VNet peering or VPN/ExpressRoute depending on the architecture.

>

For enterprise environments, I would also consider centralized hub-and-spoke networking, private DNS, route tables and Azure Firewall."

Q8. What is a Private Endpoint?

"A Private Endpoint provides a private IP address inside a VNet for accessing an Azure PaaS service. This allows traffic to stay on the private network rather than going through the public internet.

>

For example, if an application needs to access Azure Storage or Synapse securely, I can use Private Link and Private Endpoint combined with appropriate DNS configuration and access controls."

---

5. IAM / Security

Q9. How do you secure AWS IAM?

> “I follow least privilege. I avoid using root credentials for normal operations and prefer IAM roles over long-lived access keys.
>
> Permissions should be scoped to the resources and actions actually required. I also use MFA, credential rotation where applicable, CloudTrail auditing and periodic permission reviews.
>
> For workloads running on AWS, I prefer workload identities or IAM roles rather than embedding credentials in applications.”

Q10. AWS IAM vs Azure Entra ID?

> “AWS IAM manages identities, roles, policies and permissions within AWS, while Microsoft Entra ID provides identity and access management across Microsoft cloud services and applications.
>
> In AWS, IAM policies define permissions against AWS resources. In Azure, RBAC works with Entra identities to control access to Azure resources.
>
> In both cases, the principles are similar: least privilege, strong authentication, role-based access, auditing and avoiding static credentials.”

Q11. How do you manage secrets?

> “I never store production secrets directly in Git or Terraform code. Depending on the platform, I would use AWS Secrets Manager or Azure Key Vault.
>
> Applications retrieve secrets through managed identities or IAM roles wherever possible. CI/CD pipelines should use secure secret stores and masked variables.
>
> I would also rotate secrets, audit access and make sure secrets don't appear in logs or Terraform state.”

---

6. Terraform — VERY IMPORTANT

Q12. Explain Terraform workflow.

> “The normal workflow is:
>
> `terraform init` → `terraform validate` → `terraform plan` → review → `terraform apply`.
>

> Terraform maintains the desired infrastructure state in a state file. In production, I prefer remote state with locking and controlled access.

>

> I also use reusable modules, variables, outputs and environment-specific configurations rather than duplicating infrastructure code."

Q13. What is Terraform state?

> "Terraform state maps the resources defined in Terraform to the actual infrastructure. Terraform uses it to understand what exists and what changes are required.

>

> For team environments, I use remote state with locking, access control, encryption and versioning. State is sensitive because it can contain infrastructure information and sometimes sensitive values."

Q14. What happens if two engineers run Terraform simultaneously?

> "That's why remote state locking is important. The first operation acquires the lock and the second operation should wait or fail depending on the backend.

>

> Without locking, concurrent operations could modify the same state and potentially corrupt it or create inconsistent infrastructure."

Q15. Terraform vs CloudFormation?

> "Terraform is multi-cloud and uses a provider-based model, which is useful for an organization operating both AWS and Azure. CloudFormation is AWS-native and integrates deeply with AWS services.

>

> For this role, I would favor Terraform for common infrastructure automation across AWS and Azure, while CloudFormation can still be appropriate for AWS-specific requirements."

Q16. Terraform plan shows unexpected resource replacement. What do you do?

> "I don't immediately apply it. First I inspect the plan and identify which attribute is causing replacement.

>

> Then I compare the Terraform configuration with the current state and actual resource. I check whether the change is intentional, whether a provider version changed, or whether state drift exists.

>

> If necessary, I use `terraform state` operations carefully or update the configuration. In production, any destructive change requires explicit review."

---

7. CI/CD

Q17. Design a production CI/CD pipeline.

> "I would separate CI from CD.

>

CI would perform Git checkout, dependency installation, compilation or build, unit tests, static analysis, security scanning, container build and image vulnerability scanning.

>

The image would then be pushed to a trusted registry with an immutable tag.

>

CD would deploy through environments such as development, test, staging and production. Production deployment would have appropriate approvals, change controls and rollback mechanisms.

>

For Kubernetes environments, I would use Helm for packaging and ArgoCD for GitOps-based deployment."

Q18. Jenkins vs GitHub Actions vs Azure DevOps?

"All three can implement CI/CD, but their strengths and integration models differ.

>

Jenkins is highly extensible and commonly used in enterprise environments. GitHub Actions integrates tightly with GitHub repositories and provides workflow automation. Azure DevOps provides integrated repos, pipelines, boards and Azure ecosystem integration.

>

I select based on the organization's existing ecosystem, security requirements, governance and operational model."

---

8. ArgoCD — VERY IMPORTANT

Q19. What is ArgoCD?

"ArgoCD is a GitOps continuous delivery tool for Kubernetes. Git becomes the source of truth for the desired application state.

>

ArgoCD continuously compares the desired state in Git with the actual state in Kubernetes. If there's a difference, it identifies the application as OutOfSync and can synchronize the cluster with the declared configuration.

>

This gives us auditability, version control and consistent deployments."

Q20. What happens if someone manually changes a Kubernetes deployment?

"ArgoCD detects drift between the Git-defined desired state and the live Kubernetes state.

>

Depending on the sync policy, it can either alert that the application is OutOfSync or automatically reconcile it back to the Git state.

>

This is one of the main advantages of GitOps because manual production changes don't silently become the permanent configuration."

Q21. How would you implement rollback with ArgoCD?

> “Because deployments are version-controlled in Git, rollback can be achieved by reverting the configuration or Helm version to a known-good commit.
>
> ArgoCD then synchronizes the cluster with that version. I would also use Kubernetes rollout history where appropriate and make sure database changes are backward-compatible before application rollback.”

---

9. Helm

Q22. Why Helm?

> “Helm packages Kubernetes manifests into reusable charts. It allows us to parameterize environment-specific configuration using values files.
>
> For example, the same chart can deploy development, staging and production with different replica counts, resources, image tags, ingress configuration and environment variables.
>
> This reduces duplication and improves deployment consistency.”

Q23. How do you manage Helm across environments?

> “I prefer one reusable chart with environment-specific values rather than maintaining separate copies of manifests.
>
> For example:
>
> `values-dev.yaml`
>
> `values-qa.yaml`
>
> `values-prod.yaml`
>
> ArgoCD can then reference the appropriate values based on the target environment.”

---

10. Kubernetes

Q24. Explain Kubernetes architecture.

> “The control plane contains the API server, scheduler, controller manager and etcd. Worker nodes run workloads through components such as kubelet and the container runtime.
>

> The API server is the main entry point. The scheduler decides where pods should run based on resource requirements and scheduling constraints. Controllers continuously reconcile the desired state.
>
> etcd stores cluster state, so it needs to be highly available and properly backed up.”

Q25. A pod is in CrashLoopBackOff. What do you do?

> “I start with:
>
> `kubectl get pods`
>
> Then:
>
> `kubectl describe pod <pod>`
>
> and:
>
> `kubectl logs <pod> --previous`
>
> I check the container exit code, events, application logs, probes, environment variables, ConfigMaps, Secrets, resource limits and recent deployments.
>
> If the container is being OOMKilled, I check memory usage and limits. If the application is exiting due to configuration or dependency issues, I fix the root cause rather than simply restarting the pod.”

Q26. Pod is Pending. What do you check?

> “I check:
>
> `kubectl describe pod`
>
> particularly the Events section.
>
> Common causes include insufficient CPU or memory, node selectors, taints and tolerations, affinity rules, unavailable nodes or PVC issues.
>
> I also check:
>
> `kubectl get nodes`
>
> and resource availability.”

Q27. ImagePullBackOff?

> “I check the image name and tag first, then verify whether the image exists in the registry.

>

> I check registry credentials and `imagePullSecrets`, network connectivity, registry permissions and whether the Kubernetes nodes can reach the registry.

>

> `kubectl describe pod` normally gives useful events explaining the image pull failure.”

---

11. Kubernetes Networking

Q28. Explain Kubernetes Service types.

> “ClusterIP is the default and provides internal service discovery. NodePort exposes a service through a port on each node. LoadBalancer integrates with the cloud provider's load-balancing service.

>

> For HTTP/HTTPS routing, I would generally use an Ingress controller or Gateway implementation depending on the platform.”

Q29. A pod can communicate with another pod but not through the Service. What do you check?

> “I would verify the Service selector matches the pod labels and check whether endpoints exist:

>

> ```
> kubectl get svc
> ```

>

> ```
> kubectl get endpoints
> ```

>

> Then I check the target port versus container port, DNS resolution, NetworkPolicies and the application listener.

>

> If endpoints are empty, the first thing I'd investigate is the Service selector.”

---

12. Monitoring / Observability

Q30. How would you monitor a production platform?

> “I use the four major observability signals: metrics, logs, traces and events.

>

> For metrics, I would monitor CPU, memory, latency, throughput, error rate, saturation and infrastructure health.

>

> Prometheus and Grafana can provide Kubernetes and application metrics, while tools such as Azure Monitor, CloudWatch, Splunk or Dynatrace can provide broader infrastructure and application observability.

>

Alerts should be based on meaningful symptoms and SLOs rather than simply generating alerts for every metric threshold."

Q31. What are SLI, SLO and SLA?

"An SLI is the actual measurement, such as availability or request latency.

>

An SLO is the reliability target, for example 99.9% availability.

>

An SLA is the formal commitment to the customer, potentially including consequences if the commitment isn't met.

>

I use SLOs and error budgets to balance reliability with release velocity."

---

13. Production Incident Scenario

Q32. Production latency suddenly increases. What do you do?

"First I establish the impact — which services, users and regions are affected.

>

Then I check dashboards for latency, error rate, traffic, CPU, memory, saturation and dependency health.

>

I correlate the timing with recent deployments or infrastructure changes.

>

I then isolate whether the issue is application, Kubernetes, network, database or downstream dependency related.

>

For mitigation, I may rollback a deployment, scale the workload, route traffic differently or disable a problematic feature depending on the situation.

>

Once service is stable, I perform RCA and implement preventive actions such as better alerting, capacity changes or deployment safeguards."

This **Impact** → **Triage** → **Metrics/Logs** → **Isolate** → **Mitigate** → **Recover** → **RCA** → **Prevent** pattern is particularly useful for this interview.

---

14. Disaster Recovery

Q33. How would you design DR for a critical trading platform?

"I would start with business requirements: RTO and RPO.

>

> Based on those requirements, I would determine whether we need backup and restore, pilot-light, warm standby or active-active architecture.

>

> Infrastructure should be reproducible using Terraform, application deployments should be automated through CI/CD or GitOps, and data replication should be designed according to consistency requirements.

>

> I would also automate health checks, failover procedures where possible and regularly perform DR drills rather than considering DR complete just because backups exist."

Q34. What is RTO vs RPO?

> "RTO is how quickly the service needs to be restored after an outage.

>

> RPO defines the maximum acceptable amount of data loss measured in time.

>

> For example, a 15-minute RPO means we should be able to recover with no more than approximately 15 minutes of data loss."

---

15. Cost Optimization

Q35. How would you reduce AWS/Azure cloud costs?

> "I first identify the largest cost drivers using billing and utilization data.

>

> Then I look at rightsizing compute, removing unused resources, storage lifecycle policies, autoscaling, reserved or savings-plan commitments where workloads are stable, and scheduling non-production resources.

>

> For Kubernetes, I examine requests and limits, node utilization and autoscaling.

>

> I would never optimize cost by blindly reducing capacity; reliability and performance requirements must remain within the agreed SLOs."

---

16. Databricks

Q36. What is Databricks and how would you support it as a Cloud Engineer?

> "Databricks is a cloud data and analytics platform built around Apache Spark and commonly used for data engineering, analytics and machine learning workloads.

>

> From the Cloud Engineering side, I would focus on the underlying networking, identity, access control, storage integration, secrets, compute configuration, monitoring, automation and CI/CD.

>

> In an enterprise environment I would also pay attention to private connectivity, cluster policies, access controls, data security and cost management.”

Q37. How would you secure Databricks?

> “I would apply least-privilege identity and access management, private networking where required, encryption, controlled access to storage, secret management and audit logging.

>

> I would also implement cluster policies to prevent uncontrolled resource usage and integrate deployment and configuration changes with CI/CD and appropriate governance.”

---

17. Kafka / Event-Driven Architecture

Q38. Explain Kafka at a high level.

> “Kafka is a distributed event-streaming platform. Producers publish messages to topics, and consumers read those messages.

>

> Topics are divided into partitions, which provide scalability and parallelism. Consumer groups allow multiple consumers to process partitions collaboratively.

>

> For production systems, I would monitor consumer lag, broker health, throughput, partition distribution and replication.”

Q39. What would you monitor for Kafka?

> “Key metrics include consumer lag, broker CPU and memory, disk utilization, network throughput, request latency, under-replicated partitions, offline partitions and replication health.

>

> Consumer lag is particularly important because it indicates whether consumers are keeping up with producers.”

---

18. Python / Bash

Q40. Write a script to find files larger than 1 GB.

A good spoken answer:

> “On Linux, I can use `find`, for example:

>

```
find /path -type f -size +1G -exec ls -lh {} \;
```

>

> For automation, I would normally add error handling, logging and potentially exclude specific directories.”

Q41. How would you check disk utilization and alert if it exceeds 80%?

> “I would use `df` to get filesystem utilization and use Bash or Python to parse the percentage. If it exceeds the defined threshold, the script can log the event and trigger an alert through the monitoring system.”

Q42. Why use Python instead of Bash?

> “For simple Linux administration, Bash is often sufficient. For more complex automation involving APIs, JSON, error handling, cloud SDKs and structured data, I prefer Python because it provides better maintainability and testing capabilities.”

---

19. Git

Q43. Explain your Git branching strategy.

> “The exact strategy depends on the organization, but I prefer short-lived feature branches with pull requests, code review and automated validation.
>
> Protected branches prevent direct production changes. CI validates every pull request, and releases are associated with immutable commits or tags.
>
> For GitOps, the deployment repository itself becomes the auditable source of truth for the desired environment state.”

---

20. Capital Markets Scenario

Q44. You are supporting a Front Office Trading application. What are your priorities?

> “My first priorities would be availability, latency, resilience, security and observability.
>
> Trading systems can be highly sensitive to latency and downtime, so I would establish clear SLOs and monitor transaction latency, error rates, throughput and dependency health.
>
> I would also ensure strong access controls, auditability, encryption, DR and controlled production changes.
>
> From an operational perspective, incident response and change management need to be very disciplined because production changes can have significant business impact.”

---

21. Critical Production Scenario

Q45. A production deployment has caused trading transactions to fail. What do you do?

> “First I assess the impact and declare the appropriate incident severity.

>

> I check application errors, Kubernetes health, deployment status, logs, metrics and downstream dependencies.

>

> If the failure correlates directly with the latest release and rollback is safe, I would initiate rollback according to the established change process.

>

> At the same time, I communicate impact and status to stakeholders.

>

> After recovery, I perform RCA covering why testing didn't detect the problem and introduce preventive controls such as better automated tests, canary deployment, validation gates or improved monitoring.”

---

22. Security Scenario

Q46. A developer asks you to give them production admin access. What do you do?

> “I wouldn't provide unrestricted access simply for convenience. I would understand the required task and provide the minimum permissions necessary through the approved access-management process.

>

> For emergency situations, I would use controlled privileged access with auditing and time-bound permissions where the platform supports it.

>

> The objective is least privilege without blocking legitimate operational requirements.”

---

23. Infrastructure Drift

Q47. Someone manually changed production infrastructure. Terraform now shows changes. What do you do?

> “That's infrastructure drift. First I determine what was changed and why.

>

> I compare Terraform state, configuration and actual infrastructure. If the manual change is legitimate, I bring the change back into Terraform so that the code remains the source of truth.

>

> If it wasn't legitimate, I restore the intended configuration through Terraform.

>

> I would also investigate why manual changes were possible and improve access controls or process if required.”

---

24. Capacity Planning

Q48. How do you perform capacity planning?

> “I look at historical utilization and business growth trends for CPU, memory, storage, network, database connections and application throughput.
>
> I correlate infrastructure utilization with business metrics such as transaction volume.
>
> Then I forecast expected demand and define scaling thresholds with sufficient headroom.
>
> For critical systems, I also consider failure scenarios because capacity planning shouldn't only account for normal traffic.”

---

25. Blue-Green vs Canary

Q49. Explain blue-green deployment.

> “Blue-green maintains two production environments. Blue is the current version and green contains the new version.
>
> After validation, traffic is switched from blue to green. If an issue occurs, traffic can be switched back.
>
> The benefit is fast rollback, although running duplicate infrastructure can increase cost.”

Q50. Explain canary deployment.

> “Canary releases expose the new version to a small percentage of traffic first.
>
> We monitor metrics such as error rate, latency and business KPIs. If the release performs correctly, traffic is gradually increased.
>
> This reduces blast radius compared with immediately deploying to 100% of production.”

---

26. Likely Hands-On Questions

Be ready to type these without looking them up:

```
kubectl get pods -A
kubectl get nodes
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
```

```
kubectl get svc
kubectl get endpoints
kubectl get ingress
kubectl top pods
kubectl top nodes
```

Terraform:

```
terraform init
terraform validate
terraform fmt
terraform plan
terraform apply
terraform destroy
terraform state list
terraform state show <resource>
terraform output
```

Linux:

```
df -h
du -sh *
free -m
top
ps -ef
uptime
ss -lntp
netstat -tulpn
journalctl -u <service>
systemctl status <service>
systemctl restart <service>
grep
awk
sed
find
tail -f
```

Git:

```
git status
git log
git branch
git checkout
git switch
git pull
git fetch
git merge
git rebase
git cherry-pick
git revert
git reset
```

---

27. The 15 questions I would prioritize tonight

If you have limited time, concentrate heavily on these:

PriorityTopicQuestion		
■ 1	Terraform	Explain Terraform architecture/state/backend
■ 2	AWS	Design a secure production AWS environment
■ 3	Azure	Explain VNet/private endpoint/network security

■ 4	Kubernetes	Troubleshoot CrashLoopBackOff
■ 5	Kubernetes	Troubleshoot Pending pod
■ 6	ArgoCD	Explain GitOps and drift
■ 7	CI/CD	Design production CI/CD pipeline
■ 8	Security	IAM, secrets and least privilege
■ 9	Networking	Troubleshoot application connectivity
■ 10	DR	Design DR with RTO/RPO
■ 11	Observability	Design monitoring for critical platform
■ 12	Helm	Manage multi-environment deployments
■ 13	Databricks	Cloud engineering responsibilities around Databricks
■ 14	Kafka	Explain Kafka and consumer lag
■ 15	Capital Markets	Handle production trading-system outage

Your strongest positioning for this JD

Your answer pattern should repeatedly connect:

AWS + Azure → **Terraform** → **CI/CD** → **Kubernetes** → **ArgoCD/Helm** → **Security** → **Observability** → **SRE** → **Production reliability**

And don't position yourself as purely an infrastructure engineer. Your background gives you a stronger **SRE/Cloud/DevOps production-operations angle**, which fits the requirement for someone who can both **build environments and operate them reliably**.

For tomorrow, expect scenario questions such as:

> **"You have a production application running on AKS/EKS. Deployment succeeded but users are getting 5xx errors. Walk me through your troubleshooting."**

Your answer should naturally go:

Impact → **recent changes** → **ingress/load balancer** → **service/endpoints** → **pods** → **logs/events** → **dependencies** → **metrics** → **mitigation/rollback** → **RCA** → **prevention.**

That production-troubleshooting structure will be useful across a large portion of this JD.